EXPERIMENT - 8

AIM:

Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

1. Dataset Source

Dataset Name: Fashion-MNIST
File Used: Fashion-MNIST

2. Dataset Description

The dataset contains 70,000 grayscale images in 10 categories of clothing, intended to serve as a direct drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms.

- **Type:** Image Data (Numerical arrays)
- **Domain:** Computer Vision / E-commerce Retail
- **Target:** Supervised (Multiclass Classification: 10 distinct clothing categories)
- **Key Features:**

Feature	Description	Data Type
Image Input	28x28 grayscale matrices representing pixel intensities (0 to 255)	2D Numerical Array
Class 0	T-shirt/top	Categorical (Integer)
Class 1	Trouser	Categorical (Integer)
Class 2	Pullover	Categorical (Integer)
Class 3	Dress	Categorical (Integer)

Class 4	Coat	Categorical (Integer)
Class 5	Sandal	Categorical (Integer)
Class 6	Shirt	Categorical (Integer)
Class 7	Sneaker	Categorical (Integer)
Class 8	Bag	Categorical (Integer)
Class 9 (Target)	Ankle boot	Categorical (Integer)

3.Mathematical Formulation of the Algorithm

A CNN primarily uses convolution layers to extract spatial hierarchies of features, followed by pooling layers and fully connected dense layers.

- **Convolution Operation:** Applies a filter/kernel over the input image to create a feature map, computing the dot product between filter weights and input pixels.
- **Max Pooling:** Downsamples the feature maps by applying a window and taking the maximum value, reducing spatial dimensions and computation.
- **Activation Functions: * ReLU (Rectified Linear Unit):** Used in convolutional and hidden dense layers to introduce non-linearity $f(x) = \max(0, x)$.
    - **Softmax:** Used in the output layer for multiclass probability distribution. It calculates the probability of the image belonging to each of the 10 classes.
- **Optimization (Gradient Descent):** The Adam Optimizer updates weights by minimizing the Sparse Categorical Cross-Entropy Loss.

4. Algorithm Limitations

- **High Computational Cost:** CNNs require significant CPU/GPU resources and time for training, especially with deep architectures and large images.

- **Data Hungry:** Requires thousands of labeled images to generalize well and avoid overfitting.
- **Black Box Nature:** It is difficult to visualize or explain exactly *why* a CNN made a specific classification, as the learned feature maps become highly abstract in deeper layers.

5. Methodology / Workflow

1. **Load Dataset:** Import the Fashion-MNIST dataset directly from Keras.
2. **Data Preprocessing:** * **Scaling:** Normalize pixel values from 0-255 to 0.0-1.0 to speed up convergence.
    - **Reshaping:** Reshape the 2D image arrays (28x28) to add a single color channel (28x28x1), as expected by standard Conv2D layers.

3. **Building the Architecture:** Create a Sequential model consisting of 3 Convolutional blocks (Conv2D + MaxPooling2D), a Flatten layer, and Dense layers with Dropout.
4. **Model Compilation:** Use the 'Adam' optimizer and 'sparse_categorical_crossentropy' loss function.
5. **Training:** Fit the model on the training set for 10 epochs using a validation split (the test set).
6. **Performance Analysis:** Evaluate the model using Accuracy/Loss curves, a Confusion Matrix, and a Classification Report.

6. Performance Analysis

- **Accuracy/Loss Curves:** The model demonstrates rapid learning in the first few epochs. By monitoring the validation curve, we ensure the model isn't memorizing the training data (overfitting).
- **Confusion Matrix:** Used to observe misclassifications. In Fashion-MNIST, the model often struggles to differentiate between similar items, such as confusing a 'Shirt' with a 'T-shirt/top' or 'Pullover'.
- **Inference:** The CNN successfully extracts geometric and textural features (edges, patterns) to accurately categorize unseen clothing items with high accuracy.

7. Hyperparameter Tuning

- **Filters and Kernel Size:** We used 32 and 64 filters with a 3x3 kernel size to effectively capture both fine details and larger structures.
- **Dropout:** A Dropout layer with a rate of 0.5 was added before the final classification layer to randomly turn off neurons, heavily reducing overfitting.
- **Epochs:** Set to 10 to balance thorough learning with training time, while preventing the model from over-optimizing on the training data.

Code:

1. Environment Setup and Library Imports

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt

import numpy as np

import seaborn as sns

from sklearn.metrics import confusion_matrix, classification_report

print("TensorFlow Version:", tf.__version__)
```

2. Dataset Loading and Preprocessing

```
# Load the Fashion-MNIST dataset

(train_images, train_labels), (test_images, test_labels) = datasets.fashion_mnist.load_data()

# Define the class names for later visualization

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',

               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']


# Normalize pixel values to be between 0 and 1

train_images = train_images / 255.0

test_images = test_images / 255.0


# Reshape images to specify that it's a single channel (grayscale)

# CNNs expect input shape to be (batch_size, height, width, channels)

train_images = train_images.reshape((train_images.shape[0], 28, 28, 1))

test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))

print(f"Training data shape: {train_images.shape}")

print(f"Test data shape: {test_images.shape}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Training data shape: (60000, 28, 28, 1)
Test data shape: (10000, 28, 28, 1)
```

3. Data Exploration and Visualization

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    # Reshape back to 28x28 just for displaying
    plt.imshow(train_images[i].reshape(28, 28), cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```

4. Defining the Sequential CNN Model

```
model = models.Sequential()

# 1st Convolutional Block

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model.add(layers.MaxPooling2D((2, 2)))

# 2nd Convolutional Block

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D((2, 2)))

# 3rd Convolutional Block

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

# Flatten the output to feed into Dense layers

model.add(layers.Flatten())

# Fully Connected Layers

model.add(layers.Dense(64, activation='relu'))

model.add(layers.Dropout(0.5)) # Dropout to prevent overfitting

model.add(layers.Dense(10, activation='softmax')) # 10 classes

# Display the model architecture

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)
Trainable params: 93,322 (364.54 KB)
Non-trainable params: 0 (0.00 B)

5. Model Compilation and Training

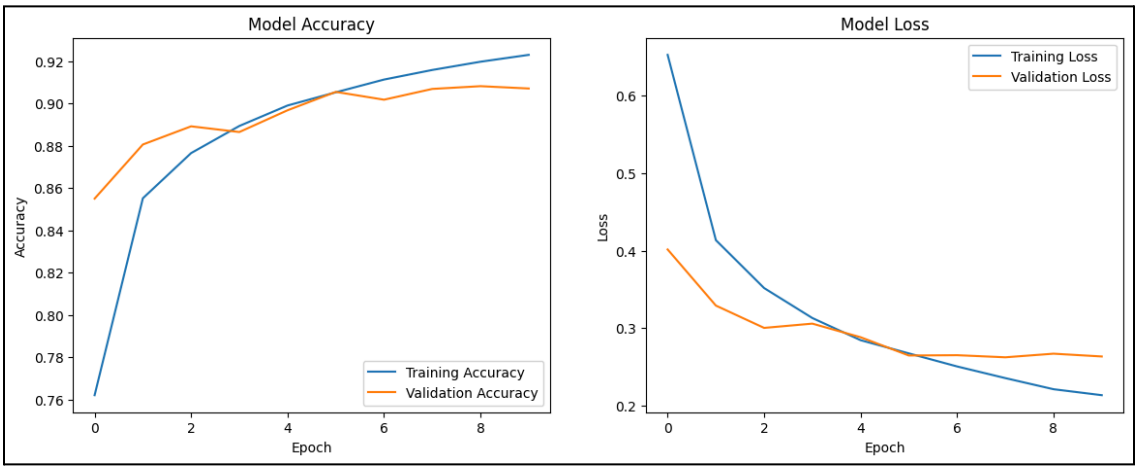
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train the model and save the history for plotting later
history = model.fit(train_images, train_labels,
              epochs=10,
              validation_data=(test_images, test_labels))
```

```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 60s 31ms/step - accuracy: 0.7620 - loss: 0.6529 - val_accuracy: 0.8550 - val_loss: 0.4017
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 60s 32ms/step - accuracy: 0.8551 - loss: 0.4137 - val_accuracy: 0.8806 - val_loss: 0.3292
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 30ms/step - accuracy: 0.8765 - loss: 0.3518 - val_accuracy: 0.8892 - val_loss: 0.3004
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 58s 31ms/step - accuracy: 0.8894 - loss: 0.3132 - val_accuracy: 0.8865 - val_loss: 0.3059
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 30ms/step - accuracy: 0.8990 - loss: 0.2846 - val_accuracy: 0.8968 - val_loss: 0.2884
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 83s 31ms/step - accuracy: 0.9053 - loss: 0.2675 - val_accuracy: 0.9055 - val_loss: 0.2648
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 30ms/step - accuracy: 0.9113 - loss: 0.2507 - val_accuracy: 0.9018 - val_loss: 0.2652
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 31ms/step - accuracy: 0.9159 - loss: 0.2356 - val_accuracy: 0.9069 - val_loss: 0.2625
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9198 - loss: 0.2212 - val_accuracy: 0.9082 - val_loss: 0.2671
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9230 - loss: 0.2136 - val_accuracy: 0.9071 - val_loss: 0.2635
```

6. Training vs. Validation Performance Analysis

```
# Create subplots for Accuracy and Loss
fig, axes = plt.subplots(1, 2, figsize=(14, 5))


# Plot Accuracy
axes[0].plot(history.history['accuracy'], label='Training Accuracy')
axes[0].plot(history.history['val_accuracy'], label='Validation Accuracy')
axes[0].set_title('Model Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend(loc='lower right')
```

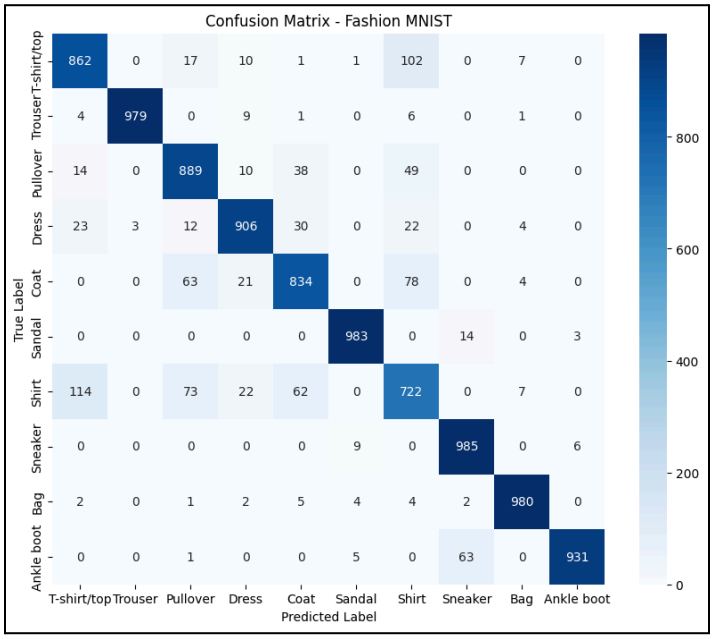
```
# Plot Loss
axes[1].plot(history.history['loss'], label='Training Loss')
axes[1].plot(history.history['val_loss'], label='Validation Loss')
axes[1].set_title('Model Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend(loc='upper right')
plt.show()
```



7. Model Validation: Confusion Matrix and Classification Metrics

```
# 1. Evaluate the overall test accuracy
test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
print(f'\nTest accuracy: {test_acc:.4f}')


# 2. Get model predictions
# The model outputs probabilities, so we take the argmax to get the predicted class index
predictions = model.predict(test_images)
predicted_labels = np.argmax(predictions, axis=1)
```

```
# 3. Create the confusion matrix

cm = confusion_matrix(test_labels, predicted_labels)


# 4. Plot the confusion matrix using Seaborn

plt.figure(figsize=(10, 8))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

            xticklabels=class_names, yticklabels=class_names)

plt.title('Confusion Matrix - Fashion MNIST')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()


# Optional: Print a detailed classification report

print("\nClassification Report:\n")

print(classification_report(test_labels, predicted_labels, target_names=class_names))
```



CONCLUSION

A Convolutional Neural Network (CNN) was successfully designed and implemented to classify the Fashion-MNIST dataset. The architecture, utilizing Conv2D and MaxPooling2D layers, effectively extracted complex spatial features and patterns from the raw image pixels, allowing the model to accurately distinguish between 10 different clothing categories. The performance analysis and confusion matrix demonstrated robust learning and strong generalization, though it highlighted occasional, expected difficulties with visually similar items (e.g., distinguishing between shirts, T-shirts, and pullovers). Overall, the experiment validates that CNNs are highly efficient and significantly superior to standard flat ANNs for computer vision and image classification tasks.